

指标	vLLM	SGLang	对比
单请求延迟 (ms)	1051.8	209.7	SGLang 快 5.0x
共享前缀耗时 (ms) 20请求并发	1071.2	378.9	SGLang 快 2.8x
QPS (8并发)	6.47	22.43	SGLang 高 3.5x
吞吐 (tok/s)	647.2	2242.5	SGLang 高 3.5x

Thinking: 为什么 SGLang 全面领先?

- 单请求延迟 (5.0x) :

vLLM 使用 --enforce-eager 且 Qwen3 默认开启 thinking 模式，生成的 token 数远多于 SGLang (SGLang 未触发 thinking)，导致延迟差异被放大

- 共享前缀 (2.8x) :

SGLang 的 RadixAttention 通过 Radix 树自动识别 20 个请求的共同前缀，KV Cache 只计算一次；vLLM 的 Prefix Caching 是块级匹配，粒度不如 Radix 树灵活

- 并发吞吐 (3.5x) :

SGLang 的缓存感知调度会优先处理命中率高的请求，叠加 RadixAttention 减少重复计算，整体 QPS 和吞吐量大幅领先

Summary (RadixAttention技术)

- 基数树 (Radix Tree) :

RadixAttention使用基数树来管理token序列和对应的KV缓存张量之间的映射。

基数树允许快速的前缀搜索、重用、插入和逐出操作。

- LRU (最近最少使用) 淘汰策略:

为了有效管理内存, 实现了LRU缓存淘汰策略。

- 缓存感知调度

通过缓存感知的调度策略, 优先处理那些最有可能重用现有KV缓存的请求, 提高处理效率和吞吐量。

通过动态地管理内存和缓存, 算法能够适应不同负载下的需求, 保持高效的运行状态。

RadixAttention技术是由LMSYS组织在它的SGLang项目中提出的

根据LMSYS组织的研究, RadixAttention可以显著提高LLM效率。在A10G GPU上使用Llama-7B和Mixtral-8x7B模型进行的测试表明, 与现有系统(如Guidance和vLLM)相比, 采用RadixAttention的SGLang系统实现了高达5倍的吞吐量提升

在大语言模型推理(特别是像 SGLang 这样的框架)中, “缓存感知调度”(Cache-Aware Scheduling)是一种为了最大化复用 KV Cache、提升吞吐量而设计的核心调度策略。

它的核心思想是:打破传统的“先来先服务”(FCFS)规则, 优先让“共享相同 Prompt 前缀”的请求连续执行, 从而避免缓存被频繁加载和换出(即“缓存抖动”)。

我们可以通过课程资料中的两个例子来直观理解:

1. 通俗比喻: 机场安检智能排队

假设机场安检通道需要根据旅客的目的地切换不同的安检设备配置。

- 传统方式(先来先服务) :
队列顺序是: 旅客A(去东京) → 旅客B(去纽约) → 旅客C(去东京) → 旅客D(去伦敦)。
安检员按顺序处理: 配好东京的设备检A → 换成纽约的设备检B → 又换回东京的设备检C。
结果: 频繁切换设备状态, 严重浪费时间。
- 缓存感知调度:
系统“预先感知”了队列中旅客的目的地, 将队列重排为: 旅客A(去东京) → 旅客C(去东京) → 旅客B(去纽约) → 旅客D(去伦敦)。
结果: 东京的安检设备状态(相当于大模型的 KV Cache)保持不动, 连续服务 A 和 C 两人, 大幅提升了整体处理效率。

2. 大模型真实场景: 批量任务处理

假设你的 AI 应用收到了 5 个并发请求。其中, 请求 1、3、4 都在使用同一个长篇的“企业知识库背景 + 评价标准”作为系统提示词(假设有 2000 个 Token), 只是最后要求评价的具体文章不同; 而请求 2、5 是毫无关联的闲聊。

- 如果不使用缓存感知调度(顺序执行 **1→2→3→4**)：GPU 处理完请求 1 后，这 2000 个 Token 的 KV Cache 存在显存中。接着处理毫无关联的请求 2，此时由于显存空间有限，请求 1 的缓存可能会被 LRU(最近最少使用)策略作为冷数据“淘汰”清空。当接着处理请求 3 和 4 时，GPU 不得不把那 **2000 个 Token** 重新计算两遍。
- 使用缓存感知调度(重排为 **1→3→4→2→5**)：调度算法会遍历等待队列，通过基数树(Radix Tree)计算每个请求与当前缓存的“前缀匹配长度”。它发现 1、3、4 共享极长的前缀，于是优先将它们放在同一批次或连续处理。这 2000 个 Token 的 KV Cache 只需在请求 1 时计算一次，请求 3 和 4 直接实现 100% 的前缀缓存命中(免费复用)，单次请求的平均耗时会呈指数级下降。